Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Übung 06: VHDL - Asserts und Verification

M.Sc. E. Trumann, M.Sc. J. Homann, Prof. Dr.-Ing. G. Payá Vayá,

28. November 2025

Parität mit Loop

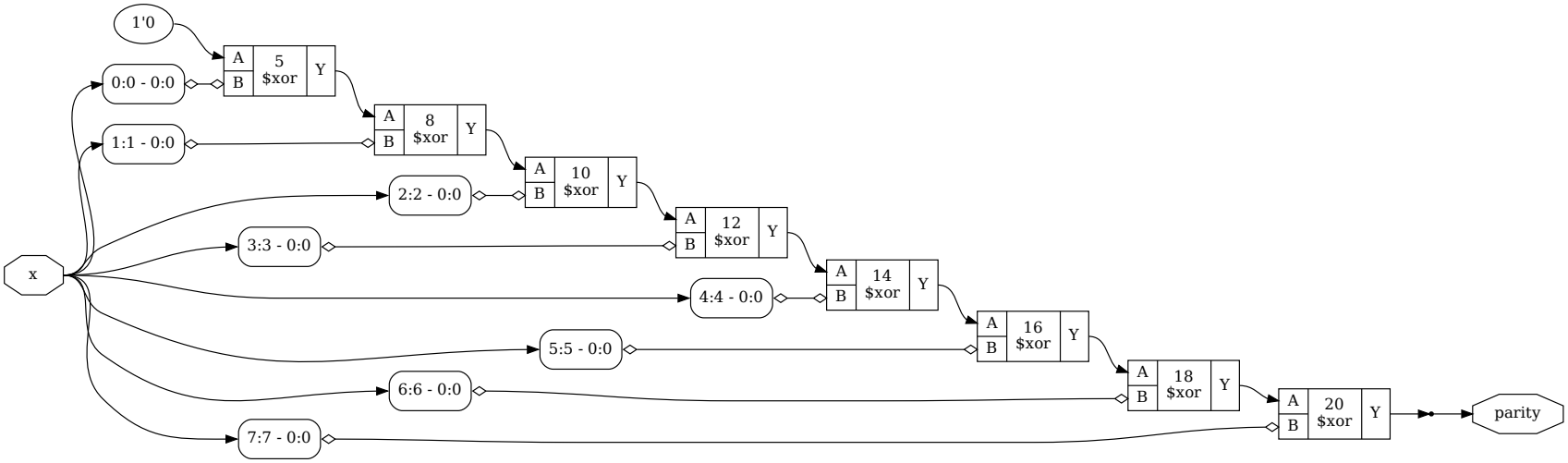
Parität:

Abhängig von der Anzahl der Einsen:

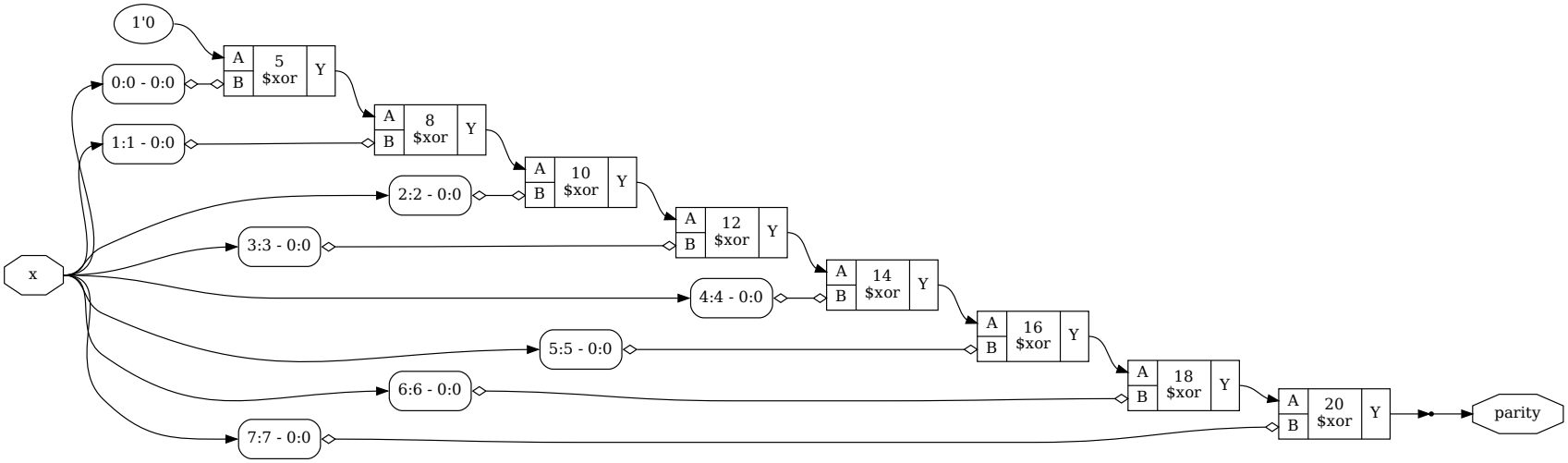
- Gerade → '0'
- Ungerade → '1'

```
function PARITY_FUNC (X : std_ulogic_vector)
        return std_ulogic is
        variable parity : std_ulogic;
begin
        parity := '0';
        for i in X'low to X'high loop
                parity := parity xor X(i);
        end loop;
        return parity;
end function PARITY_FUNC;
```

Parität mit Loop



Parität mit Loop



For-Loop beschreibt lineare Struktur → Langer kritischer Pfad

Wiederholung Signalattribute

Über Attribute (VHDL Syntax `'`) können verschiedene Eigenschaften eines Signals abgefragt werden.

- Array Grenzen (`'low`, `'high`, `'left`, `'right`)
- Array Länge (`'length`)

Beispiel: `signal X : std_ulogic_vector(7 downto 5);`

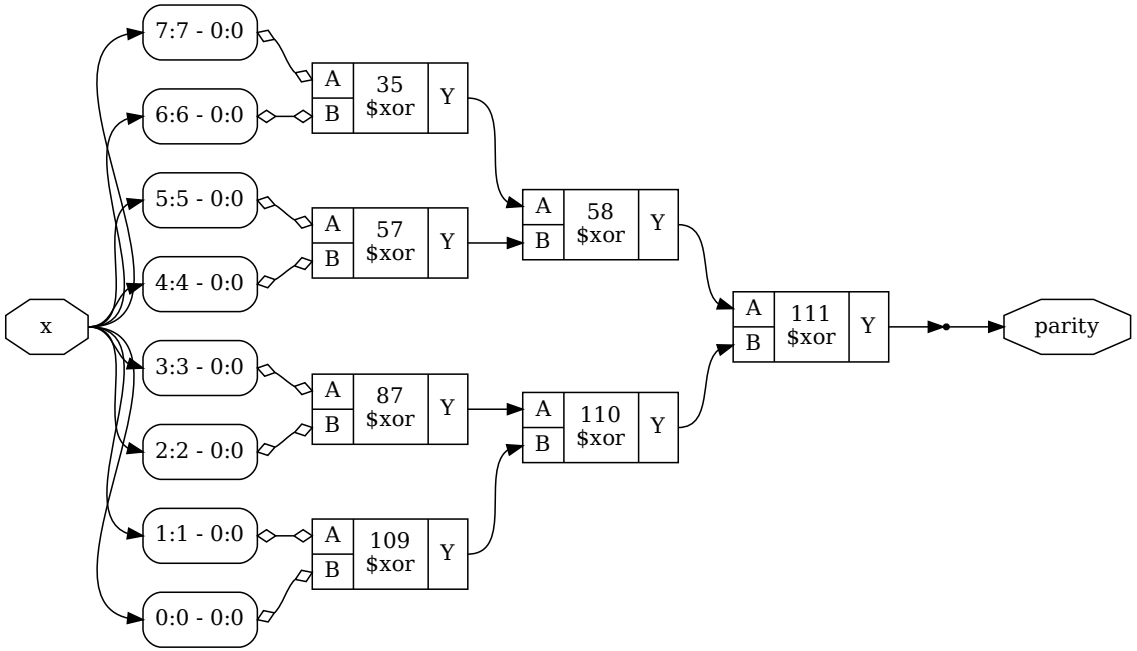
Attribut	Wert
`X'high`	7
`X'low`	5
`X'length`	3

Rekursive Parität

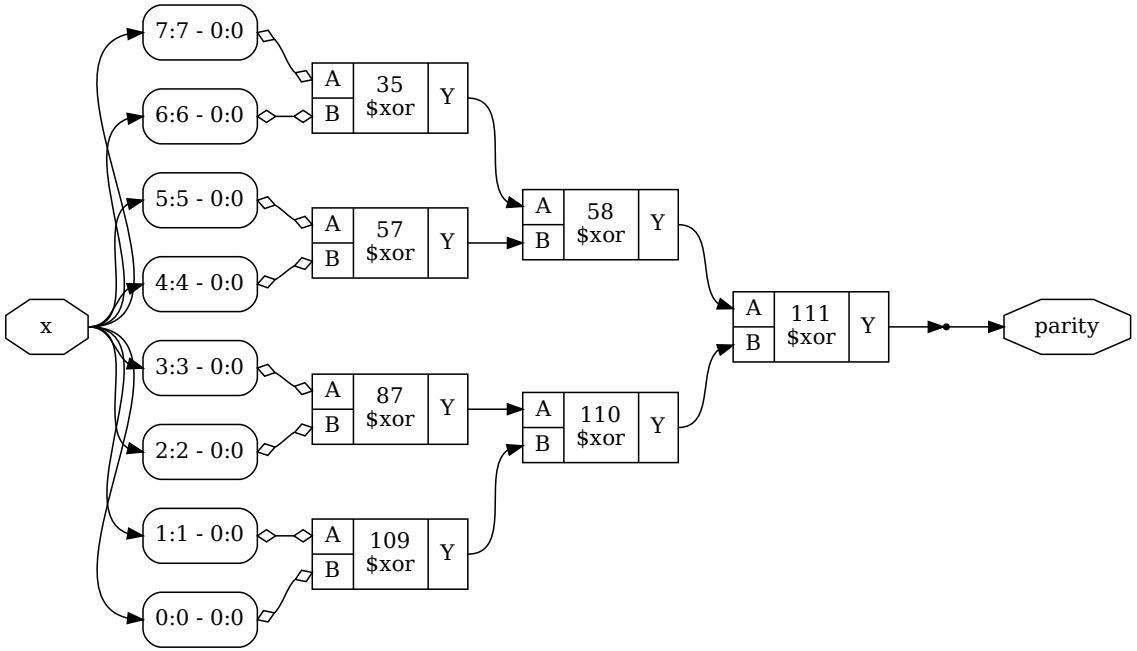
Entwickeln Sie eine Funktion, die die Parität eines `std_ulogic_vector` berechnet und als `std_ulogic` zurückgibt. Die Funktion soll sich rekursiv selbst aufrufen, die Höhe des dadurch beschriebenen Baums soll minimal sein. Verwenden Sie die Attribute `'low`, `'high` und `'length`, um die Grenzen und Länge des Eingabevektors zu bestimmen.

```
package recursive_parity_pkg is
        function PARITY (X : std_ulogic_vector) return std_ulogic;
end package;
```

Rekursive Parität



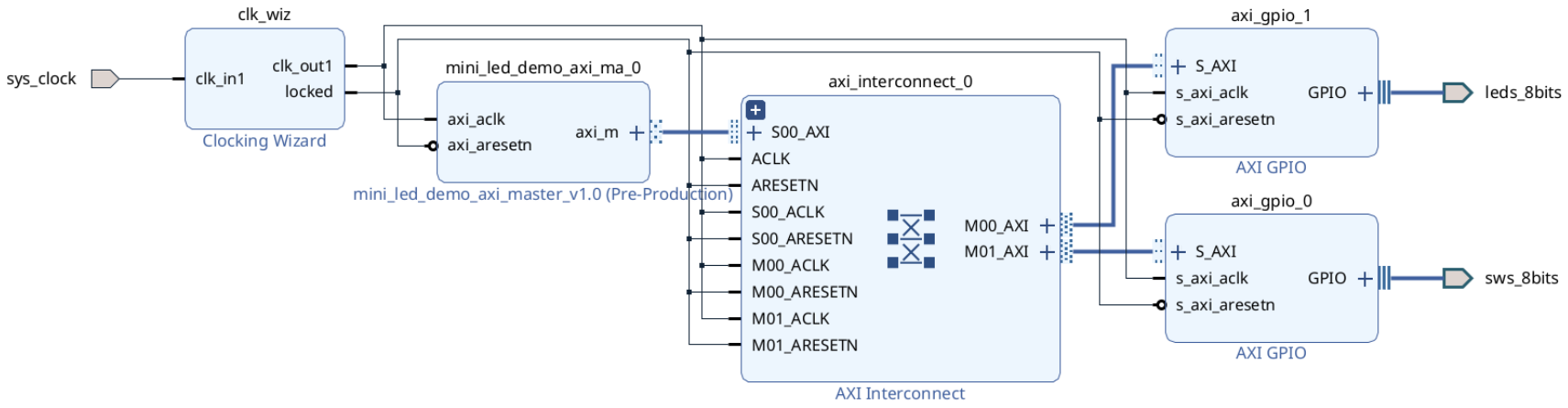
Rekursive Parität



Rekursive Funktion beschreibt Baumstruktur

$\rightarrow$ Kürzerer kritischer Pfad

IP-basierter Design Flow (e.g. Vivado)

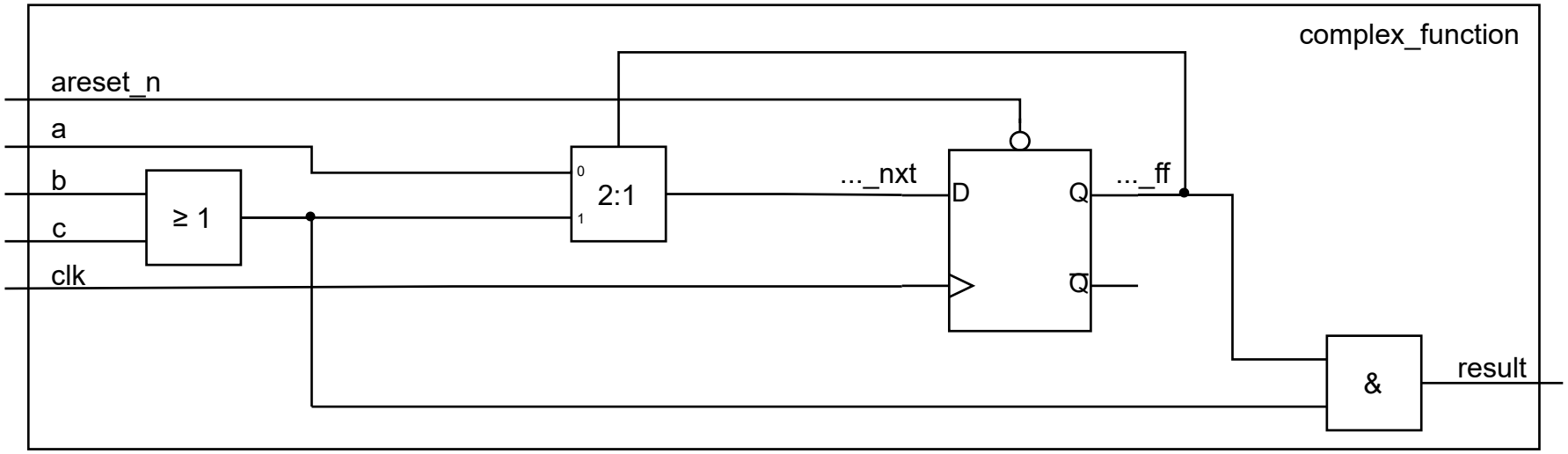


Vivado Block Diagramm

- Design wird zu großen Teilen aus wiederverwendbaren IP-Blöcken gebaut
- Spart Entwicklungszeit/-kosten
- IP-Blöcke sind getestet → weniger Risiko
- Generische Blöcke oft komplexer als benötigt

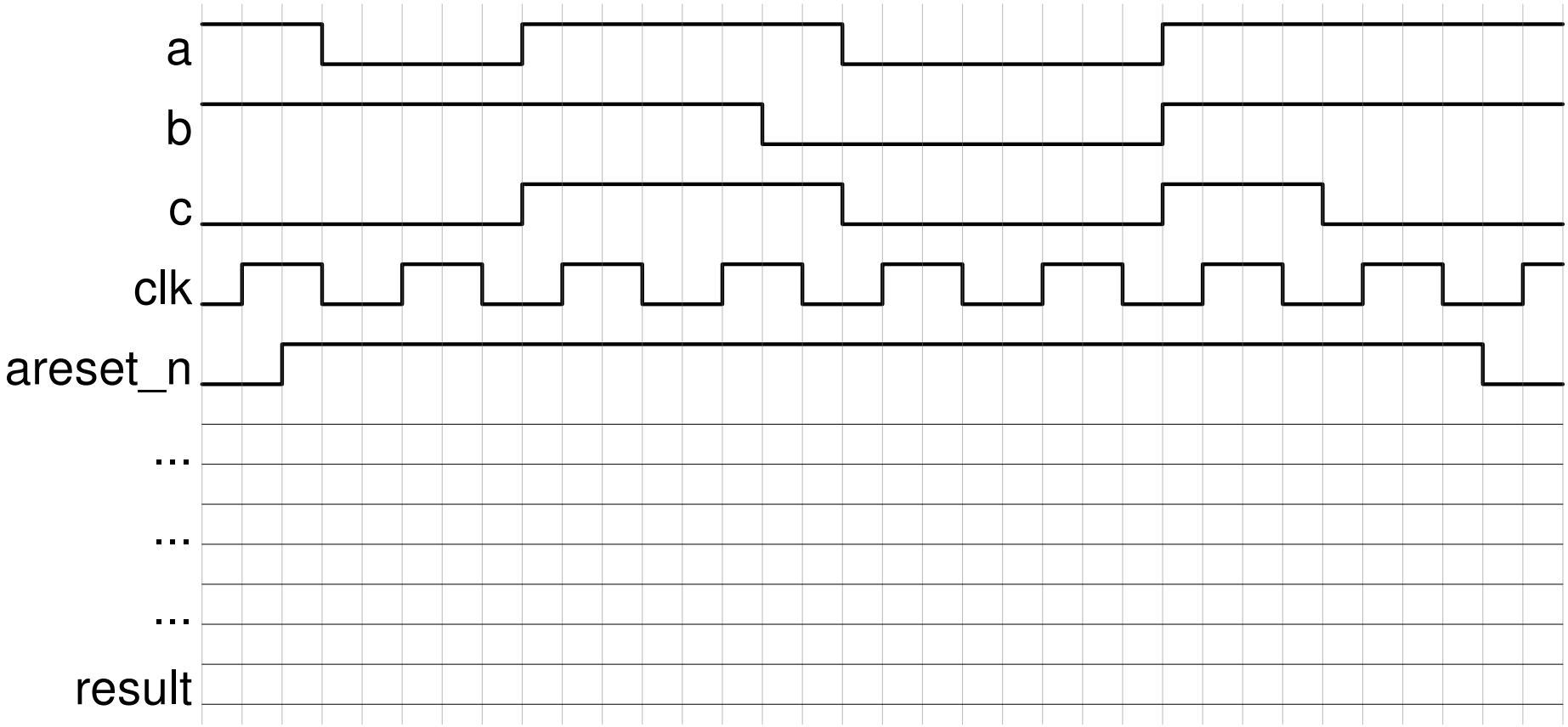
Implementierung Logik + Register

Beschreiben Sie folgende Schaltung in VHDL. Nutzen Sie zwei Prozesse (sequentiell und kombinatorische Logik getrennt).



Implementierung Logik + Register

Vervollständigen Sie das Timing-Diagramm zu dieser Schaltung



Assert

Erstellen Sie zur vorigen Schaltung (Aufgabe (2)) eine Assert-Prüfung, ob der Ausgang ’0’ ist, bei anliegendem reset (asynchron).
Erstellen Sie außerdem eine assert-Prüfung, ob der Ausgang ’0’ ist, wenn (b oder c) ’0’ ist.

Assert

Erstellen Sie für die Entity *bus_or_memory_entity* Asserts.

- Niemals darf gleichzeitig gelesen (rd_en) und geschrieben (wr_en) werden
- Das Signal 'addr' darf keine Elemente addressieren, die außerhalb der addressierbaren Elemente liegen (NUM_ELEMENTS)
- Das Signal 'valid' muss '1' sein, wenn im vorigen Takt ein Request erfolgte (req = '1'), ansonsten muss es '0' bleiben

Testbench

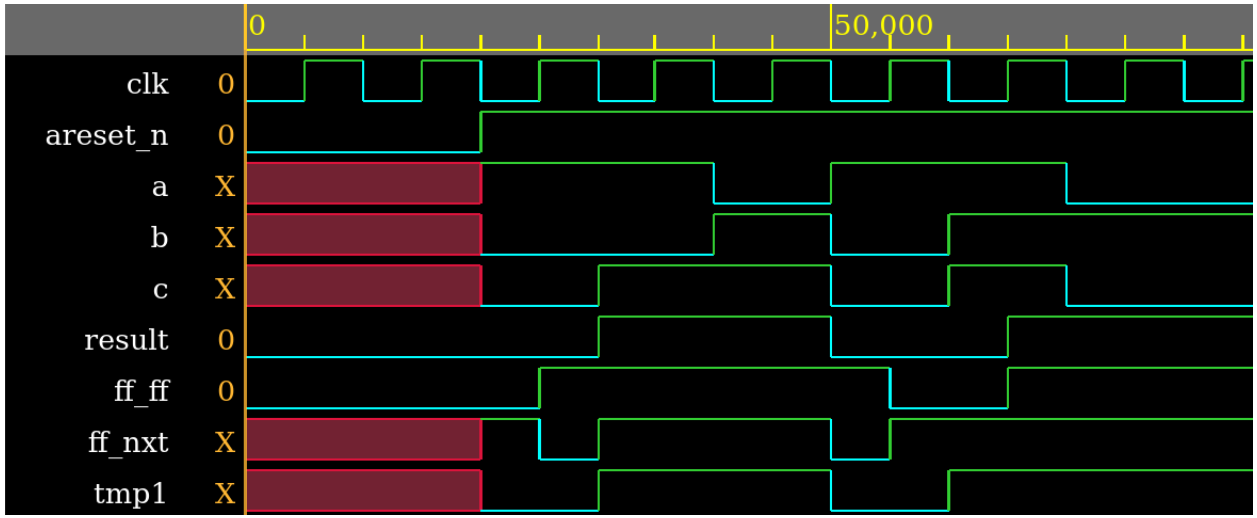
Das Modul aus Aufgabe 2) soll nun getestet werden. Die Testbench soll dafür jeweils einen Takt lang **a**, **b** und **c** anlegen. Führen Sie im Anschluss an den Reset (siehe Template) die folgende Test-Sequenz durch. Ergänzen Sie in der Tabelle zunächst die erwarteten Ergebnisse.

Test-Takt	a	b	c	result
0	1	0	0	
1	1	0	1	
2	0	1	1	
3	1	0	0	
4	1	1	1	
5	0	1	0	

Testbench

Wie hoch ist die simulierte Taktfrequenz (Signal *clk*)?

Wie hoch ist die Coverage der durchgeführten Testsequenz?
(Statement(Line)-, Branch- und Toggle-Coverage)



Testbench Lösung

Das Modul aus Aufgabe 2) soll nun getestet werden. Die Testbench soll dafür jeweils einen Takt lang **a**, **b** und **c** anlegen. Führen Sie im Anschluss an den Reset (siehe Template) die folgende Test-Sequenz durch. Ergänzen Sie in der Tabelle zunächst die erwarteten Ergebnisse.

Test-Takt	a	b	c	result
0	1	0	0	0
1	1	0	1	1
2	0	1	1	1
3	1	0	0	0
4	1	1	1	1
5	0	1	0	1

Testbench Lösung

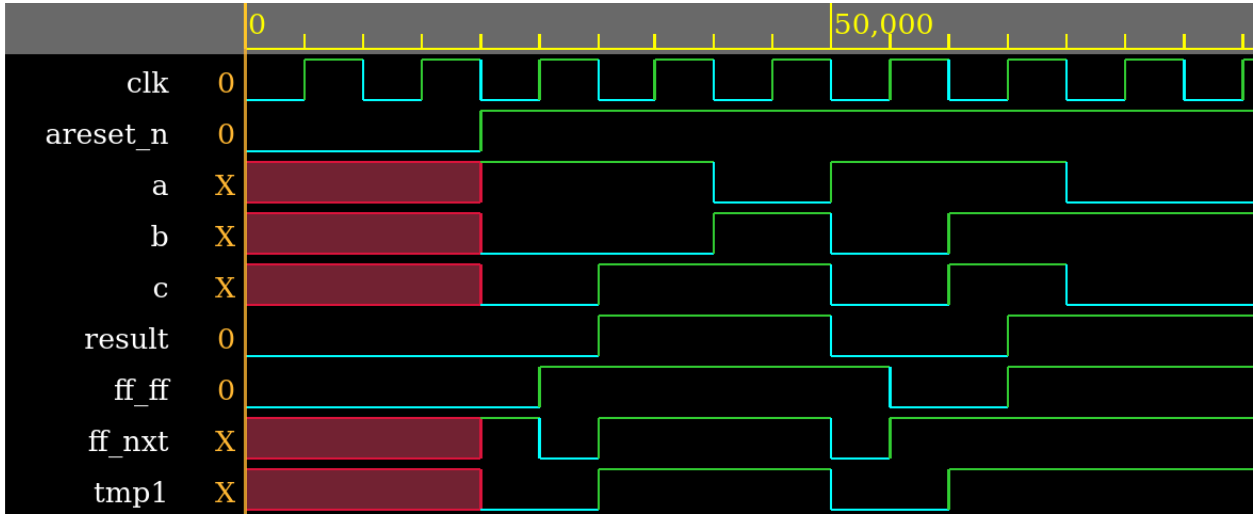
```
stimulus_process: process
begin
        -- Reset
        areset_n <= '0';
        wait for 20 ns;
        areset_n <= '1';

        -- Takt 0
        a <= 1; b <= 0; c <= 0;
        wait for 10 ns;
        assert result = '0';

        -- Takt 1
        a <= 1; b <= 0; c <= 1;
        wait for 10 ns;
        assert result = '1';
        -- Takt 2
        a <= 0; b <= 1; c <= 1;
        wait for 10 ns;
        assert result = '1';
        -- Takt 3
        a <= 1; b <= 0; c <= 0;
        wait for 10 ns;
        assert result = '0';
        -- Takt 4
        a <= 1; b <= 1; c <= 1;
        wait for 10 ns;
        assert result = '1';
        -- Takt 5
        a <= 0; b <= 1; c <= 0;
        wait for 10 ns;
        assert result = '1';
end process;
```

Testbench Lösung

Wie hoch ist die simulierte Taktfrequenz (Signal *clk*)?

$\rightarrow \frac{1}{10ns} = 100MHz$

Wie hoch ist die Coverage der durchgeführten Testsequenz?

(Statement(Line)-, Branch- und Toggle-Coverage)

$\rightarrow$Jeweils 100 %